LAPORAN

WORKSHOP SISTEM INFORMASI

WEB SERVER

(TIF330805)

SEMESTER III



BKPM Acara 10 Inheritance, Base Controller, Base Model, Repository Pattern

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pembuatan aplikasi web dengan PHP, diperlukan susunan kode yang rapi agar program lebih mudah dipahami dan dikembangkan. Penulisan fungsi yang sama secara berulang pada setiap Controller dapat membuat kode kurang efektif dan sulit dirawat. Untuk mengatasinya, konsep Inheritance digunakan agar fungsi pada BaseController dapat diwariskan kepada Controller lain. Sementara itu, Repository Pattern membantu memisahkan pengelolaan data pada database dari proses utama aplikasi sehingga struktur kode menjadi lebih teratur dan mudah dikelola.

1.2 Tujuan

1. Memahami penerapan Inheritance dalam pemrograman berorientasi objek.
2. Menggunakan BaseController sebagai class induk bagi Controller lainnya.
3. Menerapkan Repository Pattern untuk mengelola akses database.
4. Memisahkan proses pengolahan data dengan logika aplikasi.
5. Menguji fitur tambah, tampil, ubah, dan hapus data mahasiswa.

BAB 2. LANDASAN TEORI

2.1 Arsitektur Model-View-Controller (MVC)

Model-View-Controller (MVC) merupakan pola arsitektur perangkat lunak yang membagi aplikasi menjadi tiga bagian utama, yaitu Model, View, dan Controller. Model bertanggung jawab mengelola data dan berinteraksi dengan basis data, View digunakan untuk menampilkan antarmuka kepada pengguna, sedangkan Controller menangani alur permintaan dan menghubungkan Model dengan View. Penerapan MVC membantu memisahkan tanggung jawab setiap bagian sehingga kode program lebih terstruktur dan mudah dikembangkan.

2.2 Controller

Controller merupakan bagian aplikasi yang bertugas menerima permintaan pengguna, memproses alur aplikasi, serta menentukan respons yang akan diberikan. Dalam Sistem Informasi Akademik, controller dapat menangani permintaan untuk menampilkan, menambahkan, mengubah, atau menghapus data mahasiswa. Controller sebaiknya tidak dipenuhi perintah SQL secara langsung agar tanggung jawab pengaturan alur aplikasi tetap terpisah dari pengelolaan data.

2.3 Model dan Repository

Model dan repository digunakan untuk membantu mengelola data aplikasi. Repository merupakan bagian yang bertanggung jawab menjalankan operasi terhadap basis data, seperti mengambil, menambahkan, memperbarui, dan menghapus data. Pada aplikasi SI-Akademik, `MahasiswaRepository` dapat digunakan untuk mengelola data mahasiswa melalui perintah SQL. Dengan pemisahan tersebut, controller cukup memanggil metode repository tanpa perlu menuliskan query SQL secara langsung.

2.4 Pemisahan Query dari Controller

Pemisahan query dari controller merupakan praktik pengembangan aplikasi yang menempatkan perintah SQL pada bagian khusus untuk pengelolaan data. Pendekatan ini membuat controller lebih berfokus pada pemrosesan permintaan dan pengaturan alur aplikasi. Selain itu, query yang terpusat lebih mudah diperiksa,

diperbaiki, dan digunakan kembali. Pemisahan ini juga membantu mengurangi pengulangan kode ketika operasi basis data dibutuhkan oleh beberapa bagian aplikasi.

2.5 Koneksi Basis Data Menggunakan PDO

PHP Data Objects (PDO) merupakan fitur PHP yang digunakan untuk mengakses basis data. PDO menyediakan metode untuk menyiapkan dan menjalankan perintah SQL, termasuk `prepare()` dan `execute()`. Dalam arsitektur yang terstruktur, pengelolaan koneksi basis data dapat ditempatkan pada kelas khusus, kemudian digunakan oleh repository untuk menjalankan query. Cara ini membantu menghindari pembuatan koneksi berulang dan mempermudah pengelolaan akses basis data.

2.6 Implementasi Repository pada Sistem Informasi Akademik

Pada aplikasi SI-Akademik, repository dapat diterapkan untuk mengelola data mahasiswa, program studi, dan mata kuliah. Controller menerima permintaan pengguna, kemudian memanggil metode repository yang sesuai untuk mengambil atau mengubah data. Hasil pemrosesan selanjutnya diteruskan untuk ditampilkan melalui View. Dengan penerapan tersebut, tanggung jawab pengelolaan data, pemrosesan permintaan, dan penyajian antarmuka menjadi lebih jelas sehingga aplikasi lebih mudah dipelihara dan dikembangkan.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktik

A. Studi Kasus

Pada praktikum ini CRUD mahasiswa dikembangkan menggunakan Inheritance, Base Controller, Base Model, dan Repository Pattern.

3.1.1 Base Controller

```php
<?php

class BaseController
{
    protected function view($view, $data = [])
    {
        extract($data);

        require __DIR__ . '/../Views/' . $view . '.php';
    }

    protected function redirect($url)
    {
        header('Location: ' . $url);
        exit;
    }
}
```

3.1.2 Inheritance MahasiswaController

```php
require_once __DIR__ . '/../Models/MahasiswaRepository.php';

class MahasiswaController extends BaseController
{
    private const LIST_URL = BASE_URL . '/mahasiswa';

    private $repository;

    public function __construct()
    {
        $this->repository = new MahasiswaRepository(new Database());
    }
```

3.1.3 Base Model

```php
<?php

require_once __DIR__ . '/../Core/Database.php';

class Model
{
    protected $db;

    public function __construct(?Database $database = null)
    {
        $database = $database ?? new Database();

        $this->db = $database->getConnection();
    }
}
```

3.1.4 Inheritance Mahasiswa

```php
class MahasiswaRepository extends Model
{
```

3.1.5 Repository Patten

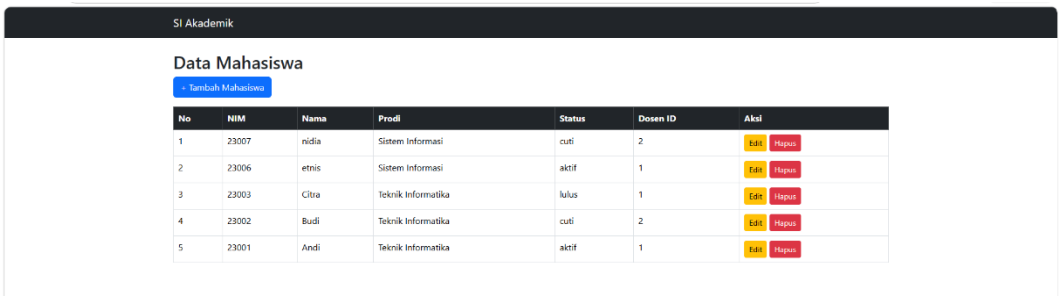
```php
public function all()
    {
        $sql = "SELECT m.id, m.nim, m.nama, p.nama AS prodi, m.status,
m.dosen_id
                FROM mahasiswa m
                LEFT JOIN prodi p ON m.prodi_id = p.id
                ORDER BY m.nim";

        $stmt = $this->db->prepare($sql);
        $stmt->execute();

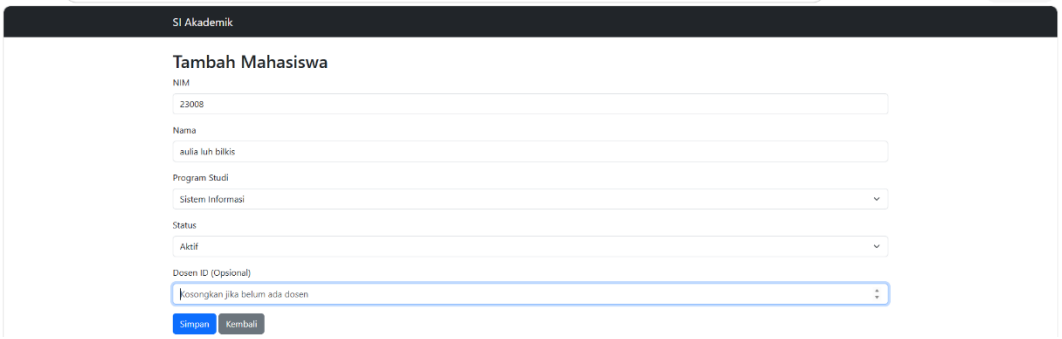
        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }
```

3.1.6 Hasil

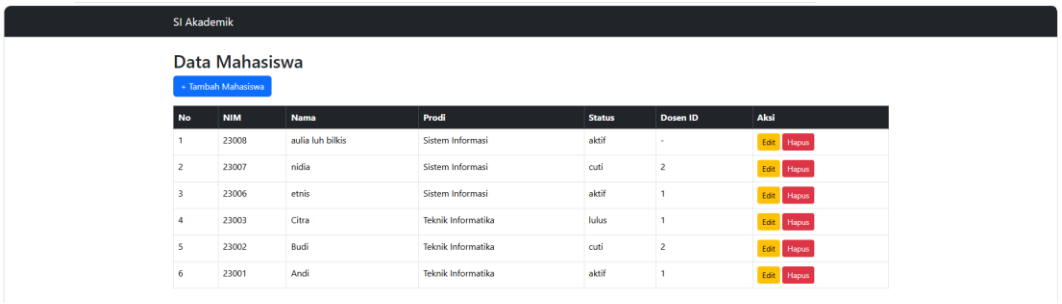
a) Data Mahasiswa berhasil di tampilkan pada halaman.



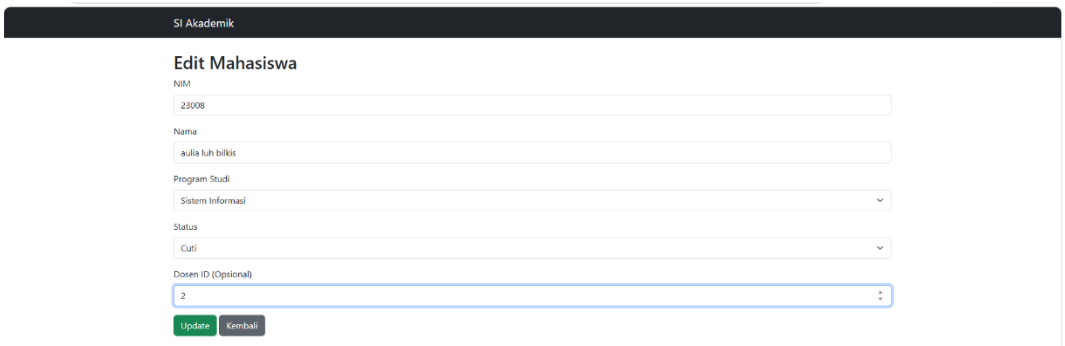
b) Menambahkan data mahasiswa.



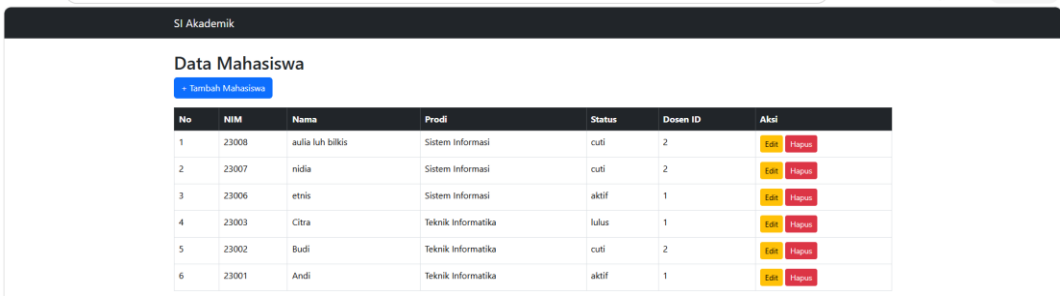
c) Data mahasiswa baru berhasil masuk ke database.



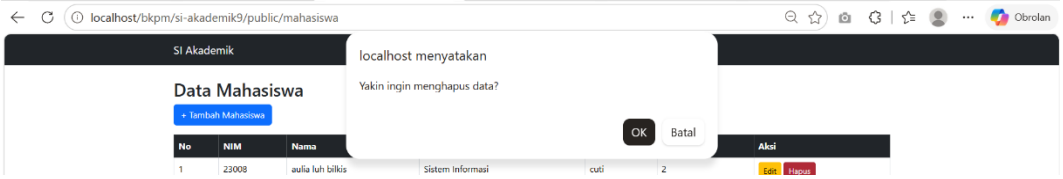
d) Update Mahasiswa



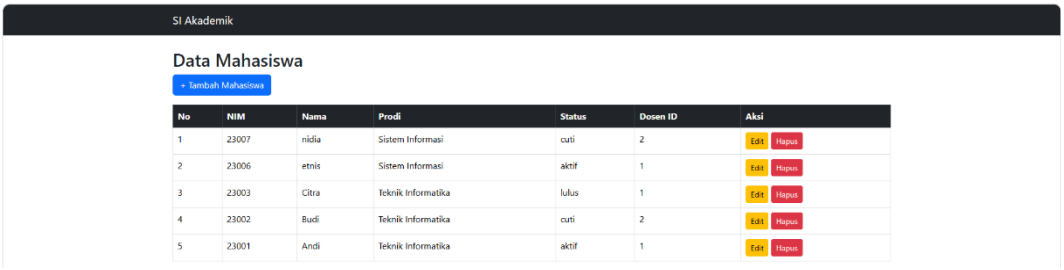
e) Data mahasiswa berhasil di update dan masuk ke dalam database.



f) Delete Mahasiswa



g) Data berhasil di hapus.



BAB 4. PEMBAHASAN

4.1 Kesimpulan

Dalam praktikum ini, aplikasi Manajemen Data Mahasiswa dibuat dengan menerapkan Inheritance dan Repository Pattern. BaseController berfungsi menyediakan method `view()` dan `redirect()` yang dapat diwariskan kepada MahasiswaController. Dengan cara ini, fungsi yang sama dapat digunakan kembali tanpa harus ditulis berulang kali.

MahasiswaRepository digunakan untuk mengelola proses pengambilan dan penyimpanan data mahasiswa ke database. Controller hanya perlu memanggil method yang tersedia di Repository tanpa harus menulis perintah SQL secara langsung. Penerapan kedua konsep tersebut membuat kode program lebih terorganisasi dan mempermudah pengembangan aplikasi.

BAB 5. PENUTUP

5.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa Inheritance berguna untuk mengurangi pengulangan kode melalui pewarisan fungsi dari class induk. Adapun RepositoryPattern membantu memisahkan proses akses database dari Controller. Kedua konsep ini mendukung pembuatan aplikasi yang lebih rapi, mudah dirawat, dan dikembangkan lebih lanjut.